DM 2: Correction

CPES ≪ Sciences des données, arts et cultures ≫ : Introduction à l'algorithmique

Antoine GAUQUIER antoine.gauquier@ens.fr
Pierre SENELLART pierre@senellart.com

11 janvier 2024

A Étude d'un algorithme de tri (10 points)

1. *(2 points)* On obtient les étapes suivantes ; à chaque étape, un appel récursif est effectué sur le sous tableau dont les indices sont indiqués. (Le tableau suivant est très détaillé, on peut se contenter de faire moins détaillé, par exemple en ne représentant pas les appels récursifs sur les tableaux de deux éléments.)

0	1	2	3	4	5	6	7	8	Indices de l'appel récursif
12	37	42	21	15	18	7	3	21	0 à 8
12	37	42	21	15	18	7	3	21	0 à 5
12	37	42	21	15	18	7	3	21	0 à 3
12	37	42	21	15	18	7	3	21	0 à 2
12	37	42	21	15	18	7	3	21	0 à 1
12	37	42	21	15	18	7	3	21	1 à 2
12	37	42	21	15	18	7	3	21	0 à 1
12	37	42	21	15	18	7	3	21	2 à 3
12	37	21	42	15	18	7	3	21	0 à 2
12	37	21	42	15	18	7	3	21	0 à 1
12	37	21	42	15	18	7	3	21	1 à 2
12	21	37	42	15	18	7	3	21	0 à 1
12	21	37	42	15	18	7	3	21	2 à 5
12	21	37	42	15	18	7	3	21	2 à 4
12	21	37	42	15	18	7	3	21	2 à 3
12	21	37	42	15	18	7	3	21	3 à 4
12	21	37	15	42	18	7	3	21	2 à 3
12	21	15	37	42	18	7	3	21	4 à 5
12	21	15	37	18	42	7	3	21	2 à 4
12	21	15	37	18	42	7	3	21	2 à 3
12	21	15	37	18	42	7	3	21	3 à 4
12	21	15	18	37	42	7	3	21	2 à 3
12	21	15	18	37	42	7	3	21	0 à 3
12	21	15	18	37	42	7	3	21	0 à 2
12	21	15	18	37	42	7	3	21	0 à 1
12	21	15	18	37	42	7	3	21	1 à 2
12	15	21	18	37	42	7	3	21	0 à 1
12	15	21	18	37	42	7	3	21	2 à 3
12	15	18	21	37	42	7	3	21	0 à 2
12	15	18	21	37	42	7	3	21	0 à 1

0	1	2	3	4	5	6	7	8	Indices de l'appel récursif
12	15	18	21	37	42	7	3	21	1 à 2
12	15	18	21	37	42	7	3	21	0 à 1
12	15	18	21	37	42	7	3	21	3 à 8
12	15	18	21	37	42	7	3	21	3 à 6
12	15	18	21	37	42	7	3	21	3 à 5
12	15	18	21	37	42	7	3	21	3 à 4
12	15	18	21	37	42	7	3	21	4 à 5
12	15	18	21	37	42	7	3	21	3 à 4
12	15	18	21	37	42	7	3	21	5 à 6
12	15	18	21	37	7	42	3	21	3 à 5
12	15	18	21	37	7	42	3	21	3 à 4
12	15	18	21	37	7	42	3	21	4 à 5
12	15	18	21	7	37	42	3	21	3 à 4
12	15	18	7	21	37	42	3	21	5 à 8
12	15	18	7	21	37	42	3	21	5 à 7
12	15	18	7	21	37	42	3	21	5 à 6
12	15	18	7	21	37	42	3	21	6 à 7
12	15	18	7	21	37	3	42	21	5 à 6
12	15	18	7	21	3	37	42	21	7 à 8
12	15	18	7	21	3	37	21	42	5 à 7
12	15	18	7	21	3	37	21	42	5 à 6
12	15	18	7	21	3	37	21	42	6 à 7
12	15	18	7	21	3	21	37	42	5 à 6
12	15	18	7	21	3	21	37	42	3 à 6
12	15	18	7	21	3	21	37	42	3 à 5
12	15	18	7	21	3	21	37	42	3 à 4
12	15	18	7	21	3	21	37	42	4 à 5
12	15	18	7	3	21	21	37	42	3 à 4
12	15	18	3	7	21	21	37	42	5 à 6
12	15	18	3	7	21	21	37	42	3 à 5
12	15	18	3	7	21	21	37	42	3 à 4
12	15	18	3	7	21	21	37	42	4 à 5
12	15	18	3	7	21	21	37	42	3 à 4
12	15	18	3	7	21	21	37	42	0 à 5
12	15	18	3	7	21	21	37	42	0 à 3
12	15	18	3	7	21	21	37	42	0 à 2
12	15	18	3	7	21	21	37	42	0 à 1
12	15	18	3	7	21	21	37	42	1 à 2
12	15	18	3	7	21	21	37	42	0 à 1
12	15	18	3	7	21	21	37	42	2 à 3
12	15	3	18	7	21	21	37	42	0 à 2
12	15	3	18	7	21	21	37	42	0 à 1
12	15	3	18	7	21	21	37	42	1 à 2
12	3	15	18	7	21	21	37	42	0 à 1
3	12	15	18	7	21	21	37	42	2 à 5
3	12	15	18	7	21	21	37	42	2 à 4
3	12	15	18	7	21	21	37	42	2 à 3
3	12	15	18	7	21	21	37	42	3 à 4
3	12	15	7	18	21	21	37	42	2 à 3
3	12	7	15	18	21	21	37	42	4 à 5
3	12	7	15	18	21	21	37	42	2 à 4
3	12	7	15	18	21	21	37	42	2 à 3
3	12	7	15	18	21	21	37	42	3 à 4
3	12	7	15	18	21	21	37	42	2 à 3

0	1	2	3	4	5	6	7	8	Indices de l'appel récursif
3	12	7	15	18	21	21	37	42	0 à 3
3	12	7	15	18	21	21	37	42	0 à 2
3	12	7	15	18	21	21	37	42	0 à 1
3	12	7	15	18	21	21	37	42	1 à 2
3	7	12	15	18	21	21	37	42	0 à 1
3	7	12	15	18	21	21	37	42	2 à 3
3	7	12	15	18	21	21	37	42	0 à 2
3	7	12	15	18	21	21	37	42	0 à 1
3	7	12	15	18	21	21	37	42	1 à 2
3	7	12	15	18	21	21	37	42	0 à 1
3	7	12	15	18	21	21	37	42	Tableau final

2. *(2 points)*

```
from math import ceil, floor

def tri(A, start = 0, end = None):
    if end == None:
        end = len(A)
    n = end - start
    if n < 2:
        pass
    elif n == 2:
        if A[start] > A[start + 1]:
            A[start], A[start + 1] = A[start + 1], A[start]
    else:
        sup =  int(ceil(2/3.*n))
        inf = int(floor(2/3.*n))
        tri(A, start, start + sup)
        tri(A, end - inf, end)
        tri(A, start, start + sup)
```

3. *(2 points)* On fait une preuve par récurrence sur la taille n du tableau.

   Cas de base. Si $n \leqslant 2$, le tableau est trié avec un tri par insertion, dont on sait qu'il est correct.

   Cas récursif. Supposons que pour tout $n' < n$, l'algorithme trie correctement les tableaux de taille n', et montrons que c'est également le cas pour un tableau de taille $n \geqslant 3$.

   Il suffit de montrer que pour deux éléments a et b du tableau tels que $a < b$, dans le tableau résultant de l'application de l'algorithme de tri, a est avant b.

   On décompose le tableau en trois sous-tableaux disjoints : T_1 comprenant les $n - \lfloor 2n/3 \rfloor$ premiers éléments ; T_2 les éléments dont l'indice est entre $n - \lfloor 2n/3 \rfloor$ et $\lceil 2n/3 \rceil - 1$; T_3 pour les éléments dont l'indice est supérieur ou égal à $\lceil 2n/3 \rceil$.

   Le premier appel récursif trie (correctement par hypothèse de récurrence) la concaténation de T_1 et T_2 ; le deuxième la concaténation de T_2 et T_3 ; le troisième la concaténation de T_1 et T_2 à nouveau.

   Après le premier appel récursif, tous les éléments de T_1 et T_2 qui doivent aller dans T_3 une fois triés seront placés dans T_2 : en effet, ces éléments sont plus grands que $\lceil 2n/3 \rceil$ autres éléments, donc ne peuvent être plus petits que tous les éléments de T_2 après ce premier appel récursif.

   Après le second appel récursif, tous les éléments initialement dans le tableau et qui doivent aller dans T_3 une fois triés seront donc placés dans T_3, dans l'ordre.

Le dernier appel récursif s'assure que les autres éléments du tableau, ceux de T_1 et T_2 sont bien triés.

4. *(1 point)* $T(n) = 3T\left(\frac{2n}{3}\right) + O(1)$. En effet, il y a trois appels récursifs sur des tableaux de taille $\frac{2n}{3}$ plus des tests et d'autres opérations (traitement du cas de base) qui se font en temps constant.
5. *(2 points)* On est dans un cas d'application du théorème maître avec $a = 3$, $b = \frac{3}{2}$ et $f(n) = O(1)$. On calcule $c = \log_b a = \log_3 1.5 \approx 2.71$. On a $f(n) = O(n^{c'})$ avec $c' = 1 < c$ donc on est dans le premier cas du théorème maître.
   On déduit donc : $T(n) = \Theta(n^c)$ (avec $c \approx 2.71$).
6. *(1 point)* Cette complexité est bien pire que la complexité des algorithmes optimaux de tri comme le tri fusion ($\Theta(n \log n)$) ou même des algorithmes les plus simples, comme le tri par insertion ($\Theta(n^2)$). Ce tri ne présente pas vraiment d'autre avantage donc n'a pas spécialement d'intérêt.
   Pour référence, cet algorithme a été appelé stooge sort *ou* tri faire-valoir.

B Compteur binaire (10 points)

7. *(0.5 point)* $n_{\max}(k) = \sum_{i=0}^{k-1} 1 \times 2^i = \frac{2^k - 1}{2 - 1} = 2^k - 1$ (somme d'une série géométrique).
8. *(4 points)*

```python
import numpy as np

class CompteurBinaire:
    def  __init__(self, k):
        self.__k =  k
        self.__tab = np.zeros(k, dtype=bool)
        self.__nb_set_bits = 0

    def getK(self):
        return self.__k

    def nbSetBits(self):
        return self.__nb_set_bits

    def increment(self):
        i = 0
        while i < self.__k and self.__tab[i] == 1:
            self.__tab[i] = 0
            self.__nb_set_bits -= 1
            i += 1
        if i < self.__k:
            self.__tab[i] = 1
            self.__nb_set_bits += 1
```

```
    def decrement(self):
        i = 0
        while i < self.__k and self.__tab[i] == 0:
            self.__tab[i] = 1
            self.__nb_set_bits += 1
            i += 1
        if i < self.__k:
            self.__tab[i] = 0
            self.__nb_set_bits -= 1

    def getInt(self):
        result = 0
        for i in range(self.__k-1, -1, -1):
            result *= 2
            result += self.__tab[i]
        return result
```

9. *(1.5 point)* Le constructeur a une complexité de $\Theta(k)$ car il nécessite d'initialiser un tableau de taille k. Les méthodes `increment` et `decrement` ont, dans le pire des cas, une complexité de $\Theta(k)$ car ils parcourent l'ensemble des bits. Telles qu'implémentées, les méthodes `getK` et `nbSetBits` sont en $O(1)$, car elles retournent des constantes. (Une implémentation plus directe aurait conduit à une méthode `nbSetBits` en $\Theta(n)$.)

10. *(3 points)* On prend comme fonction de potentiel pour une structure de compteur binaire O le nombre de bits valant 1, multiplié par une constante α à définir : $\varphi(O) = \alpha \times O.\texttt{nbSetBits()}$.
On considère une séquence de m incrémentations à partir d'un objet fraîchement construit de taille k. Soit $O_0, O_1, \ldots, O_m$ cette séquence et $c_1, \ldots, c_m$ la complexité de chacune des incrémentations.
En suivant la méthode de potentiel, on pose $\hat{c}_i = c_i + \varphi(O_i) - \varphi(O_{i-1})$. En regardant le code de la méthode `increment`, on s'aperçoit qu'il y a autant d'itérations de la première boucle que de différence du nombre de bits mis à 1 entre la structure initiale et la structure finale (à un près, en fonction de si la condition du `if` est vérifiée). En d'autres termes,
$$c_i \leqslant \beta \times (O_{i-1}.\texttt{nbSetBits()} - O_i.\texttt{nbSetBits()}) + \gamma$$
pour certaines constantes β et γ.
Donc $\hat{c}_i = c_i + \varphi(O_i) - \varphi(O_{i-1}) \leqslant (O_{i-1}.\texttt{nbSetBits()} - O_i.\texttt{nbSetBits()}) \times (\beta - \alpha) + \gamma$. En choisissant $\alpha = \beta$, on obtient $\hat{c}_i \leqslant \gamma$.
On a donc
$$\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \hat{c}_i - \varphi(O_m) + \varphi(O_0) \leqslant m\gamma$$
car $\varphi(O_0) = 0$ et $\varphi(O_m) \geqslant 0$.
La complexité amortie de m incrémentations est donc :
$$\frac{1}{m} \sum_{i=1}^{m} c_i \leqslant \gamma = O(1).$$

11. *(1 point)* On considère la séquence où l'on fait m fois une décrémentation puis une incrémentation, à partir d'un objet fraîchement construit. On s'aperçoit que chaque décrémentation va faire passer d'un compteur rempli de 0 à un compteur rempli de 1, et chaque incrémentation va faire passer d'un compteur rempli de 1 à un compteur rempli de 0. Donc la complexité de chacune des opérations, sans exception, sera de $\Theta(k)$. La complexité amortie de la séquence, dans ces conditions ne peut être que $\Theta(k)$ également.